

Finwiser × NVIDIA — Technical Fit Note

Entity: Finwiser Technologies LLP (LLPIN ACT-7598) · **Date:** 28 May 2026 **Companion to:** the pitch deck and the full module-level Technical Fit Report.

Positioning

Finwiser is an AI-led, compliance-first personal finance decisioning platform for India, built around the Account Aggregator framework. The platform ingests consented multi-source financial data, normalises it into a unified financial ledger, applies transformer-based transaction categorisation, and produces explainable, goal-based decisions that drive a SEBI-registered Investment Adviser practice operating under the Finwiser brand.

Finwiser is a regulated, Account-Aggregator-native AI decisioning system: transformer-based transaction categorisation, GPU-amenable analytics over ~14,000 fund time-series, and explainable financial reasoning — moving to privacy-preserving in-country inference for India's salaried middle class.

Five concrete workloads in the production codebase are GPU-relevant. This note describes each, its current implementation, the specific NVIDIA component it maps to, and the existing benchmark/harness that makes a comparison defensible.

1. Private / open-weight LLM inference for financial reasoning

Where it lives: Streaming advisor Q&A, AI diagnostic question generation (JSON-structured), categorisation enrichment (free-text → category), and an agentic orchestrator that composes multi-step financial reasoning.

Current state: All LLM calls route through a single OpenAI client pointed at the US endpoint — `gpt-4o-mini` for most paths and `gpt-4o` for the agentic orchestrator. A privacy gateway scrubs PII, drops blocked fields, and buckets amounts/dates before egress; every consumer is wrapped by a circuit breaker with template fallback.

Why GPU / NVIDIA fit: Financial PII must remain on Indian infrastructure. The privacy gateway, breakers, streaming machinery, and structured-output handling are already in production — the only swap point is the LLM endpoint itself, a `baseUrl` change in one file (`utils/openai-client.js:39`). **Llama-3.1-8B-Instruct or Nemotron via NIM** (OpenAI-compatible `/v1/chat/completions`) is a drop-in replacement for `gpt-4o-mini` - class calls. The agentic `gpt-4o` path is the harder migration (larger model, stricter parity requirements) and would be benchmarked last under a configurable model env var.

Why this is real, not aspiration: An offline eval harness (`scripts/eval-prompts.js`) with a 5-axis rubric, LLM-as-judge scoring, and golden-prompt fixtures already exists — we can replay every prompt through a NIM endpoint and measure parity directly.

The sovereign-inference angle: Today the privacy gateway *must* strip identifiers and withhold amounts because data crosses to the US. With in-region NIM, that stripping becomes defence-in-depth rather than a precondition — the advisor can be given richer context and answers improve.

2. Transformer / NLI-based transaction categorisation

Where it lives: The P2B tier of our 7-layer categorisation cascade (hard-match guard → P0 known-contact → P1 EWMA consensus → P2 lexicon → **P2B zero-shot NLI** → P4 history → P5 cadence → P6 fallback). P2B is the only ML tier; everything else is deterministic by design for audit replay.

Current state: Local zero-shot NLI cascade via `@xenova/transformers` (ONNX/WASM, CPU-bound): MobileBERT-MNLI (fast) escalates to BART-large-MNLI (accurate). **The accurate model is currently throttled** — a `MEMORY_GUARD_RSS_MB = 2200` rule (`zero-shot-model.service.js:35`) silently skips Layer 2 entirely under host memory pressure. CPU inference on the accurate model runs ~8 seconds per call.

Why GPU / NVIDIA fit: P2B is the accuracy ceiling on novel merchants but is currently bottlenecked by host RAM, not methodology. **TensorRT + Triton Inference Server** brings BART-large-MNLI from ~8 s to tens of ms, eliminates the silent fallback to the less-accurate model, and removes the memory guard. Triton's dynamic batching fits our batch-oriented evaluation path.

Why this is real: The cascade has built-in instrumentation — `getCascadeStats()` exposes the Layer 2 skip rate (`layer2Skipped`) — and 21 golden profile fixtures plus an AA replay harness let us measure the accuracy lift cleanly, not hypothetically.

3. GPU-accelerated analytics across ~14,000 Indian mutual fund schemes

Where it lives: Nightly/weekly peer-ranking cron over the AMFI universe (~14K schemes). Compute spans trailing returns, CAGR, daily log returns, annualised volatility ($\sqrt{248}$ trading-day convention), Beta, Jensen's Alpha, geometric-mean Sharpe, Information Ratio (raw + deflated), quarterly-stepped rolling beat rate, 4-factor Returns-Based Style Analysis (RBSA), and Direct-vs-Regular implicit expense ratios.

Current state: Pure-JavaScript implementation. The style-analysis OLS is built from scratch — $X^T X$, Gaussian elimination, 5×5 inversion, all in triple-nested loops — and runs **twice per equity fund × ~10,000 equity funds**. Cron wall-clock is **10–30 minutes**; recompute happens weekly because daily isn't viable inside that envelope.

Why GPU / NVIDIA fit: This is the strongest GPU candidate in the entire product. Same math over ~14K independent NAV series — embarrassingly parallel, columnar, batchable.

- **RAPIDS cuDF** handles columnar analytics: log returns, vol, beta, rolling windows, and `groupby().quantile([.25, .5, .75])` for distribution stats become single vectorised calls.
- **cuPy / cuML** batches the OLS normal equations: stack all funds into a 3-D tensor and solve every fund's system in one `cupy.linalg.solve`. ~20,000 tiny inversions is exactly where batched GPU linear algebra wins.

Win: 10–30 min → low-single-digit minutes, enabling **daily** recompute instead of weekly, with headroom for richer analytics within the same window.

Caveat (stated honestly): Part of the cron is I/O-bound (sequential Mongo NAV fetches per scheme). The correct architecture is bulk-load NAV columns once → GPU batch → bulk-write — which also fixes the sequential-fetch antipattern. We don't claim "we can't run today" — we claim **headroom for daily cadence and richer analytics**.

4. Monte Carlo simulations for goal and SIP confidence modelling

Where it lives: Retirement-fund planner and goal-feasibility services — probability-of-success modelling for whether a chosen SIP will meet a goal corpus given assumptions about return distributions and volatility.

Current state: Single-threaded Box-Muller draws in JavaScript, capped at **1,000 iterations** because more blocks the Node event loop (`retirement-fund-planner.service.js:704-775`). Confidence is currently surfaced through a clamped linear `confidenceDelta` heuristic, not a real probability of success.

Why GPU / NVIDIA fit: I.i.d. paths, embarrassingly parallel. **RAPIDS / CuPy** runs 10^5 – 10^6 paths per goal in milliseconds, replacing the linear heuristic with a real CDF over outcome distributions. Per-user latency budget allows it comfortably.

Win: A statistically defensible probability-of-success figure replaces a heuristic that was capped by single-thread compute, not by methodology — a direct improvement to the user-facing "will I make it?" answer.

5. Natural-language rationale generation over deterministic advice codes

Where it lives: The "Next Best Rupee" allocator emits **8 structured rationale codes** (`must_have_bias` , `emergency_underfunded` , `high_urgency` , `within_policy_bands` , `stable_surplus` , `high_priority` , `progress_momentum` , `default_recommendation`). Goal-feasibility emits its own `REASON_CODES` set. The decision logic itself is deterministic — a multiplicative utility over hand-tuned constants, fully replayable for SEBI audit.

Current state: Codes are surfaced via templated prose. Functional, but not personalised, English-only, and doesn't adapt to the user's financial vocabulary.

Why GPU / NVIDIA fit: A small instruction-tuned model via **NIM (7–8B class)** expands a code plus its numeric context into one personalised Hindi/English sentence. **Crucially, the codes remain the source of truth and audit trail — the LLM only renders, never decides.** This preserves SEBI explainability (a learned model on the *decision* would be a compliance defect) while materially improving UX.

Why this is real: The 8 codes are already enumerated in production code; the calculation audit log captures `{params, results, reconciliation}` per decision with a 90-day TTL. A NIM renderer plugs in at the presentation layer without touching the decision engine or the audit chain.

What Finwiser is deliberately NOT doing with GPU

Stating this explicitly is the cleanest credibility signal — the workloads above aren't pitch fiction; they're the parts of the system where GPU *genuinely* fits, called out against a much larger surface area where it doesn't.

- **Tax classification & LTCG math** — statutory rules. A predictive tax model would be a compliance bug, not a feature.
- **42 × 7 mutual-fund suitability matrix** — curated, not learned. A model that overrides it defeats its purpose.
- **Counterparty resolution** — exact-match (VPA → phone → name-prefix → HMAC fingerprint). Embeddings here would reduce auditability.
- **Risk profiling, dual-language calc parity, XIRR, per-lot tax engine** — must remain bit-deterministic for SEBI replay; small-data per-user paths where PCIe transfer dwarfs the compute.

Proposed Innovation Lab experiment (60-day DGX Cloud bundle)

A single bundled benchmark, defensible because every component has an existing CPU baseline and golden-fixture harness already in the codebase:

1. **Self-hosted LLM vs gpt-4o-mini** — Llama-3.1-8B NIM behind the same `baseURL`, replay golden-prompt fixtures through the 5-axis eval harness. Apples-to-apples.
2. **TensorRT-compiled NLI on Triton** — replay 21 golden profile fixtures + AA replay harness; measure removal of the `layer2Skipped` degradation.
3. **RAPIDS batch analytics** — replay one full peer-ranking job over the ~14K-scheme NAV table; target 10–30 min → low-single-digit minutes; golden-file parity test ($\pm 0.5\%$ vs Value Research / Morningstar reference values).
4. **GPU Monte Carlo** — 1K-iter CPU vs 10^6 -path GPU on the same goal inputs, with an explicit kill/keep calibration criterion.

Each component maps to a harness, fixture set, or CPU baseline that **already exists in the production codebase** — results are defensible, not hypothetical.

Supporting verification facts

- **Entity:** Finwiser Technologies LLP · LLPIN ACT-7598 · PAN AALFF0177G
- **Incorporation:** 24 December 2025, MCA Central Registration Centre, Manesar (Form 16, LLP Act 2008 §12(1))
- **DPIIT Startup Recognition:** DIPP240313 (issued 22 January 2026; Industry: Finance Technology, Sector: Personal Finance; valid through 23 December 2035)
- **Sister concern:** SEBI Investment Adviser INA000021331 (granted 04 November 2025) + FIU/Account Aggregator implementation certificate FIU/V2.0/0335 (10 December 2025, Suma Soft — Sahamati-empanelled, ReBIT NBFC-AA API Spec v2.0.0) — held by the founder's proprietorship under the Finwiser brand
- **Trademark:** FINWISER (TM App. 7409395, Classes 36 + 42); usage NOC granted to the LLP
- **Production codebase:** ~50k LOC Node backend (`Finwiser_backend/`) + Flutter cross-platform frontend; MVP in closed testing
- **Eval & replay harnesses in repo:** `scripts/eval-prompts.js`, `services/ingestion/AA/aa-replay-harness.js`, 21 golden profile fixtures in `test/fixtures/profiles/`

Branding guardrail (per Inception terms): Acceptable usage — *"Finwiser is a member of NVIDIA Inception."*
No claims of NVIDIA endorsement, validation, or backing will be made.